React do tutorial ao mercado

Minicurso de React para o mercado de trabalho

Tutorial e produção são mundos diferentes

React de tutorial

- O foco é fazer funcionar, custe o que custar
- useState e useEffect para tudo
- Componentes de 600 linhas
- Estilo solto, sem padrão

React de mercado

- O foco é manter e prever o que o código faz
- Responsabilidades separadas: UI, estado e cache
- Acessibilidade, tolerância a falhas e performance
- Ecossistema modular e padronizado

Como o ecossistema amadureceu

Fase 1: o velho oeste	CSS global, Bootstrap, jQuery misturado e Redux com dezenas de arquivos por tela
Fase 2: CSS-in-JS e pacotes gigantes	Styled-components, MUI, abstração demais e dependência cega de pacotes enormes
Fase 3: o padrão atual	Headless, código dentro do projeto, design tokens e estado do cliente separado do estado do servidor

O mercado aprendeu com a dor dos legados.

O que quem revisa seu PR enxerga

Previsibilidade	TypeScript estrito e Zod, sem any e sem surpresa em runtime
Desacoplamento	Componente visual não conhece regra de banco, regra de negócio não conhece tag HTML
Facilidade de manutenção	Outra pessoa assume sua tarefa sem precisar refatorar tudo

Código sênior é aquele que um júnior consegue ler e manter.

O que vamos ver hoje

UI e primitivos

- Tailwind, shadcn/ui
- Radix UI, Base UI, Lucide

Formulários e estado

- React Hook Form, Zod
- Zustand, TanStack Table

Dados do servidor

- Axios com interceptors
- TanStack Query

Gráficos

- ECharts vs. ApexCharts

Arquitetura

- MVVM aplicado ao React

Escala

- Design System
- Module Federation

Estilização: como era e como é

Como era (styled-components)

- Custo em runtime: o CSS é gerado via JavaScript
- Sem suporte a React Server Components
- Nomes inventados (ContainerWrapper, SubBoxText) cansam
- CSS vazando e classes duplicadas

Como é (Tailwind CSS)

- Zero runtime: tudo resolvido no build
- Tokens fixos de margem, cor e fonte vindos do Design
- Autocomplete na IDE acelera a entrega

Bibliotecas de UI: a virada

Modelo monolítico (MUI, Ant, Mantine)

- Componente fechado dentro do node_modules
- Customizar vira briga com classe interna e ! important
- Um major update pode quebrar metade do sistema

Modelo shadcn/ui

- O código é copiado para src/components/ui
- Controle total, com Tailwind e TypeScript
- Sem pacote único gigante, sem medo de breaking change

Quando são 10 sistemas: Design System e UI Library

Design System	O conceito amplo: tokens no Figma, regras de acessibilidade, tom da marca e documentação
UI Library interna	A implementação em código: um pacote privado (@empresa/ui) usado por todas as equipes
Por que investir	Consistência de marca entre produtos e menos retrabalho: componente bom construído por uma equipe serve para todas

Como construir essa lib sem reinventar a roda? Primitivos.

Primitivos headless: a engenharia invisível

O que é

- Traz lógica, comportamento e acessibilidade (WAI-ARIA)
- Não traz nenhum estilo

Radix UI

- Base do shadcn, bem maduro
- Focus trap, portais e navegação por teclado

Base UI

- Resposta do time do MUI para o mundo headless

Acessibilidade não é opcional: exigência legal, processo e leitor de tela.

Ícones: consistência e paridade com o Figma

Lucide

- Padrão do ecossistema shadcn
- Leve, traço unificado, ótimo para dashboards

Phosphor

- 6 pesos por ícone (thin a duotone)
- Ideal para estado ativo: weight="fill"

Hugeicons

- 4.000+ ícones, stroke, twotone e solid
- Em alta em SaaS e fintechs

Tree-shaking: o bundler leva só os ícones que você importa, não a biblioteca inteira. Estilo via Tailwind: size-5, stroke-[1.5].

Formulários em produção: React Hook Form

Como era (componentes controlados)

- Um useState por input ou um objeto central
- Cada letra digitada re-renderiza o formulário inteiro
- Telas com 30 ou 40 campos travavam em máquina modesta

Como é (React Hook Form)

- Inputs não controlados, lê o DOM só quando precisa
- Re-render cirúrgico: só o campo digitado reage
- Integra com shadcn via Form, FormField e FormControl

Zod: validação em runtime e tipagem

O problema do TypeScript sozinho

- Só existe em desenvolvimento, some no navegador
- Uma interface User não impede dado inválido no formulário
- Antes: ifs e regex soltos, duplicados com os tipos

Zod

- Um schema declarativo com todas as regras
- z.infer extrai o tipo TypeScript do schema
- Fonte única da verdade: sem tipo manual separado

Zustand: estado global sem exagero

O que vinha antes

- Redux clássico: actions, reducers e sagas para mudar um isSidebarOpen
- Context API abusada: qualquer mudança re-renderiza tudo embaixo

Por que o Zustand pegou

- Menos de 2KB, sem Provider
- Seletores granulares: useStore(s => s.user)
- Regra de ouro: só estado de UI (modal, tema, sidebar). Cache de API não

Tabelas: TanStack Table + shadcn

Como era

- Libs que amarravam lógica e CSS
- Um botão customizado dentro da célula exigia gambiarra
- Montar na mão com <table> vira caos

TanStack Table (headless)

- Cuida de ordenação, paginação, seleção, filtros e colunas fixas
- Zero CSS: o visual vem do Table do shadcn com Tailwind
- Em B2B e SaaS, tabela é boa parte do trabalho

TanStack Table ou AG Grid?

TanStack Table + shadcn

- Headless, com liberdade visual total
- Ótimo para dashboards, portais e SaaS
- Bundle pequeno, gratuito e open-source

AG Grid ("Excel do navegador")

- Feito para alta densidade: 100 mil+ linhas
- Virtualização, pinning, pivot e edição inline prontos
- Community grátis, Enterprise com licença paga

Axios: ainda faz sentido?

Instância central	api.ts com baseURL, timeout e headers padrão
Erros padronizados	Rejeita a Promise em 4xx e 5xx; o fetch trata 500 como sucesso
Cancelamento	AbortController simples, sem chamadas fantasmas ao trocar de aba
JSON automático	Sem chamar .json() em cada requisição

Interceptors: segurança invisível

Request interceptor	Anexa Authorization: Bearer <token> sem ninguém lembrar em cada tela
Response interceptor	Ao receber 401, pausa a fila, chama /refresh-token, atualiza o storage e repete as requisições
Impacto	O usuário não é deslogado no meio de uma tarefa importante

TanStack Query: chega de useEffect para API

O hábito antigo

- Três useState (data, loading, error) e um useEffect gigante por tela
- Race condition, fetch duplicado e nenhum cache
- Dado do banco tratado como se fosse do React

Server state com useQuery

- O dado do servidor é um espelho temporário
- Cache: voltou para a tela, já aparece
- Stale-while-revalidate: mostra o cache e atualiza em segundo plano
- Refetch ao focar a janela ou reconectar

Mutações e atualização otimista

useMutation	Para POST, PUT e DELETE, com isPending, isSuccess e isError
Invalidação declarativa	Cadastrou um usuário? invalidateQueries(['users']) e a lista se atualiza sozinha
Optimistic update	A interface reage antes da API responder (like, checkbox); se falhar, faz rollback e avisa o erro

Gráficos: SVG ou Canvas?

SVG

- Cada barra e ponto é um nó no DOM
- Nítido e fácil de animar
- Pesa quando há muitos dados

Canvas / WebGL

- Uma única tag <canvas>, desenhada via JavaScript
- Aguenta milhões de pontos sem criar nós no HTML
- Dashboards reais têm dados dinâmicos, tooltip e resize

ApexCharts

Pontos fortes

- Visual bonito de fábrica (gradientes, tooltips)
- react-apexcharts, configuração por props
- Responsivo sem esforço

Quando usar

- Painéis executivos de SaaS (MRR, churn, vendas)
- Dados já agregados, até ~1.000 registros
- Limite: baseado em SVG, perde fôlego com volume

Apache ECharts

Pontos fortes

- Canvas, WebGL e SVG: 100 mil+ pontos a 60 FPS
- DataZoom, brush e seleção para drill-down
- Candlestick, mapas geoespaciais e heatmap

Custo

- Curva de aprendizado maior
- Objeto options extenso e imperativo
- Compensa em monitoramento, cotações e logística

Qual escolher?

Volume de dados	Até 1.000 pontos: ApexCharts. De mil a centenas de milhares: ECharts
Estética pronta	ApexCharts ganha com folga
Análise avançada (zoom, mapa, heatmap)	ECharts é imbatível

TanStack Query busca → função formata as séries → gráfico renderiza.

O "Componente Deus"

O que mora num UserDashboard.tsx

- Chamadas de Axios ou TanStack Query
- Estado de modal, filtro e paginação
- Validação e conversão de valores
- 300 linhas de JSX, Tailwind e ícones

O que isso custa

- Quase impossível testar a regra sem montar a tela
- Merge conflict toda vez que duas pessoas mexem nela
- Ninguém sabe onde acaba a regra e começa o layout

MVVM adaptado ao React

Model

- Tipos, schemas Zod e serviços do Axios
- Não conhece React

ViewModel

- Custom hook (useUserManagementViewModel)
- API, estado, validação e handlers
- Zero JSX

View

- Componente com Tailwind e shadcn
- Só consome o que o hook expõe
- Sem lógica

MVVM na prática

ViewModel (useDashboardViewModel.ts)

- useQuery para buscar métricas
- Estado do período (month, year)
- useMutation para exportar
- Retorna { metrics, isLoading, changePeriod, exportReport }

View (DashboardView.tsx)

- const { metrics, isLoading, changePeriod } = useDashboardViewModel()
- Renderiza Card, ECharts e botões
- Zero lógica: só liga o onClick

Monorepo e a UI Library da empresa

Cenário	10 aplicações (banking, portal do parceiro, backoffice, checkout) e várias equipes ao mesmo tempo
Estrutura (Turborepo, pnpm, Nx)	apps/ com portal-web, admin e checkout; packages/ com @empresa/ui, @empresa/tokens e @empresa/utils
Storybook	Catálogo onde design e dev validam variações antes de ir pra produção

Versionando a UI Library

Patch (1.0.1)	Correção de bug, sem mexer nas props
Minor (1.1.0)	Prop ou recurso novo, compatível com o anterior
Major (2.0.0)	Breaking change no contrato
Ferramentas	Changesets para changelog; registry privado (GitHub Packages, CodeArtifact, Nexus)

Como atualizar o botão sem quebrar 8 aplicações em produção?

Module Federation e micro-frontends

A dor	Build de 25 minutos; se a equipe de Perfil sobe um bug, o Checkout inteiro fica parado
Host (shell)	Carrega cabeçalho, rodapé e cuida da autenticação
Remotes	Cada equipe publica sua parte, isolada
Em runtime	O host baixa o JS do remote pela rede e integra as telas

Nem tudo são flores

Lei de Conway	Micro-frontend resolve problema de organização, não é requisito técnico inicial
CI/CD	Deploys independentes pedem uma esteira madura
Dependências	Host e remotes precisam compartilhar a mesma versão do React
Ambiente local	Rodar vários serviços e micro-frontends consome muita memória

Comece com um monólito, escale com monorepo e só use micro-frontends quando a coordenação entre equipes doer.